

Military Asset Management System

Project Documentation

1. Project Overview

The Military Asset Management System is a modular operations dashboard designed to help command, logistics, and base-level personnel monitor, assign, transfer, and expend critical military assets across multiple bases. The initial framework focuses on core operational transparency and accountability rather than a full production-grade enterprise platform.

Assumptions

- Assets tracked include vehicles, weapons, ammunition, and other equipment categories.
- The system is intended for command-level oversight and operational planning across a small-to-medium multi-base environment.
- Data can be seeded and used for demonstration and continuing development.

Limitations

- The current implementation is an initial framework and not a fully hardened production enterprise solution.
- Authentication is demonstration-oriented rather than a full JWT/OAuth system.
- Role enforcement relies on request headers in the backend and does not yet include MFA or secure session management.
- Database connectivity must be configured in the backend environment for live operational use.

2. Tech Stack and Architecture

Backend: Node.js + Express

- REST API layer for the dashboard, purchases, transfers, assignments, and expenditures.
- Middleware-based RBAC enforcement.
- Audit logging for operational accountability.

Frontend: React + Vite

- Fast UI development environment with dashboard-driven views.
- Pages for dashboard, purchases, transfers, assignments, and inventory tracking.
- Fetches data from the backend API using a shared API URL helper.

Database: PostgreSQL + Drizzle ORM

- Core tables provide structured relational storage for bases, inventory, transfers, and assignments.
- Drizzle ORM supports schema-driven development and database query management.
- PostgreSQL is suitable for transactional operations, role-aware reporting, and audit retention.

Architecture

- Frontend requests are sent to `http://localhost:5000/api`.
- Backend validates role and base access for each route.
- When the database is valid and connected, data is read from the database; otherwise, fallback demo data is returned.

3. Data Models and Schema

Core entities and relationships:

- bases: id, name, commander, created_at